

5.11 专题：*k*-means 聚类

在前面几节中，我们探索了一种无监督机器学习模型：降维。下面将介绍另一种无监督机器学习模型：聚类算法。聚类算法直接从数据的内在性质中学习最优的划分结果或者确定离散标签类型。

虽然在 Scikit-Learn 或其他地方有许多聚类算法，但最简单、最容易理解的聚类算法可能还得算是 ***k*-means** 聚类算法了，在 `sklearn.cluster.KMeans` 中实现。首先还是先输入标准程序包：

```
In[1]: %matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns; sns.set() # 绘图风格
import numpy as np
```

5.11.1 *k*-means 简介

k-means 算法在不带标签的多维数据集中寻找确定数量的簇。最优的聚类结果需要符合以下两个假设。

- “簇中心点”（cluster center）是属于该簇的所有数据点坐标的算术平均值。
- 一个簇的每个点到该簇中心点的距离，比到其他簇中心点的距离短。

这两个假设是 *k*-means 模型的基础，后面会具体介绍如何用该算法解决问题。先通过一个简单的数据集，看看 *k*-means 算法的处理结果。

首先，生成一个二维数据集，该数据集包含 4 个明显的簇。由于要演示无监督算法，因此去除可视化图中的标签（如图 5-110 所示）：

```
In[2]: from sklearn.datasets.samples_generator import make_blobs
X, y_true = make_blobs(n_samples=300, centers=4,
                        cluster_std=0.60, random_state=0)
plt.scatter(X[:, 0], X[:, 1], s=50);
```

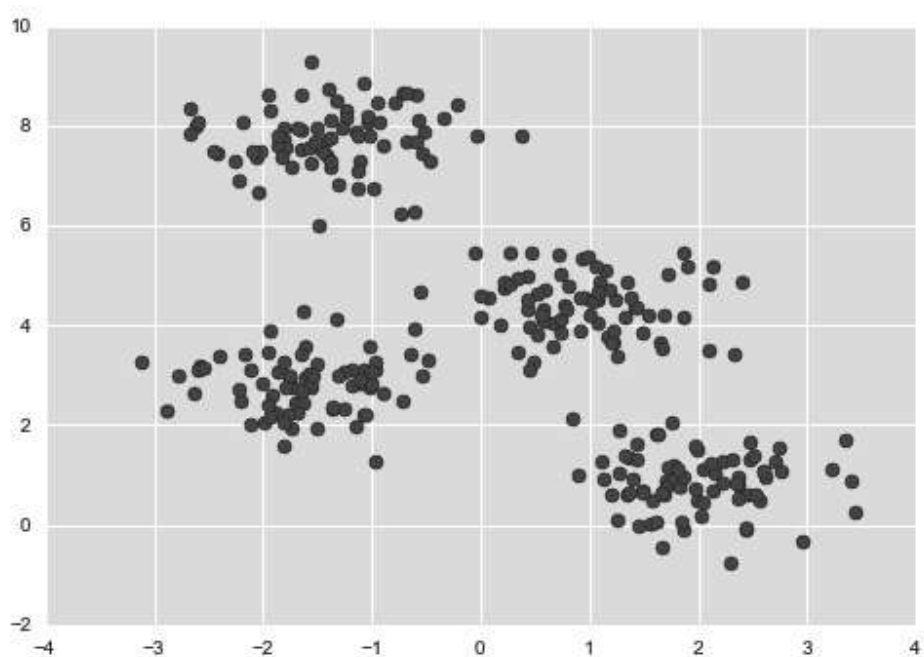


图 5-110：数据聚类

通过肉眼观察，可以很轻松地挑选出 4 个簇。而 *k*-means 算法可以自动完成 4 个簇的识别工作，并且在 Scikit-Learn 中使用通用的评估器 API：

```
In[3]: from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=4)
kmeans.fit(X)
y_kmeans = kmeans.predict(X)
```

下面用带彩色标签的数据来展示聚类结果。同时，画出簇中心点，这些簇中心点是由 *k*-means 评估器确定的（如图 5-111 所示）：

```
In[4]: plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, s=50, cmap='viridis')

centers = kmeans.cluster_centers_
plt.scatter(centers[:, 0], centers[:, 1], c='black', s=200, alpha=0.5)
```

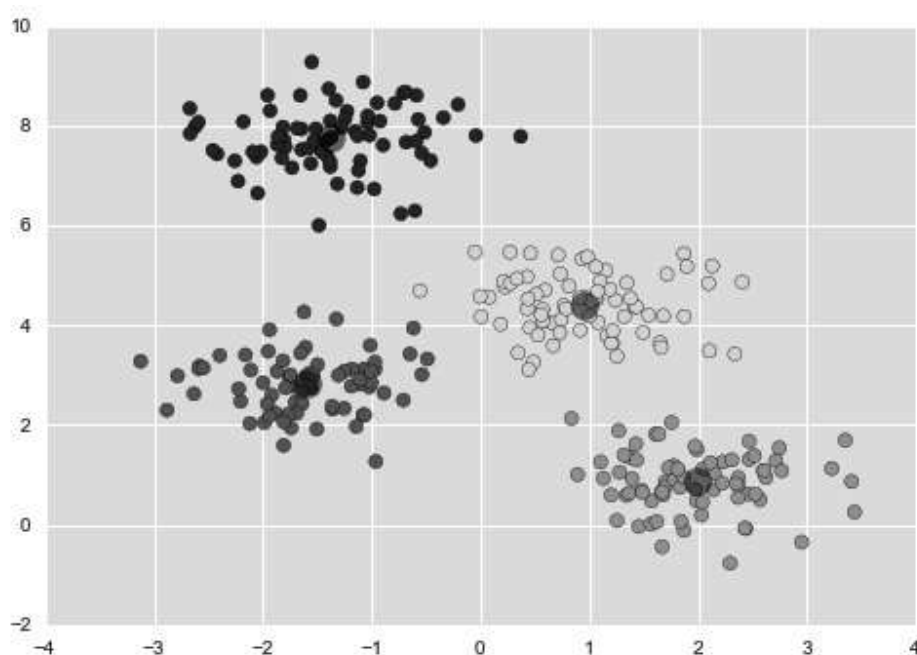


图 5-111: **k-means** 簇中心点和用不同颜色区分的簇

告诉你个好消息，**k-means** 算法可以（至少在这个简单的例子中）将点指定到某一个类，就类似于通过肉眼观察然后将点指定到某个类。但你可能会好奇，这些算法究竟是如何快速找到这些簇的，毕竟可能存在的簇分配组合方案会随着数据点的增长而呈现指数级增长趋势，如果要做这样的穷举搜索需要消耗大量时间。好在有算法可以避免这种穷举搜索——**k-means** 方法使用了一种容易理解、便于重复的期望最大化算法取代了穷举搜索。

5.11.2 **k-means**算法：期望最大化

期望最大化（expectation-maximization, E-M）是一种非常强大的算法，应用于数据科学的很多场景中。**k-means** 是该算法的一个非常简单并且易于理解的应用，下面将简单介绍 E-M 算法。简单来说，期望最大化方法包含以下步骤。

- (1) 猜测一些簇中心点。
- (2) 重复直至收敛。
 - a. 期望步骤（E-step）：将点分配至离其最近的簇中心点。

b. 最大化步骤（M-step）：将簇中心点设置为所有点坐标的平均值。

期望步骤（E-step 或 Expectation step）不断更新每个点是属于哪一个簇的期望值，最大化步骤（M-step 或 Maximization step）计算关于簇中心点的拟合函数值最大化对应坐标（ argmax 函数）——在本例中，通过简单地求每个簇中所有数据点坐标的平均值得到了簇中心点坐标。

关于这个算法的资料非常多，但是这些资料都可以总结为：在典型环境下，每一次重复 E-step 和 M-step 都将会得到更好的聚类效果。

将这个算法在图 5-112 中可视化。

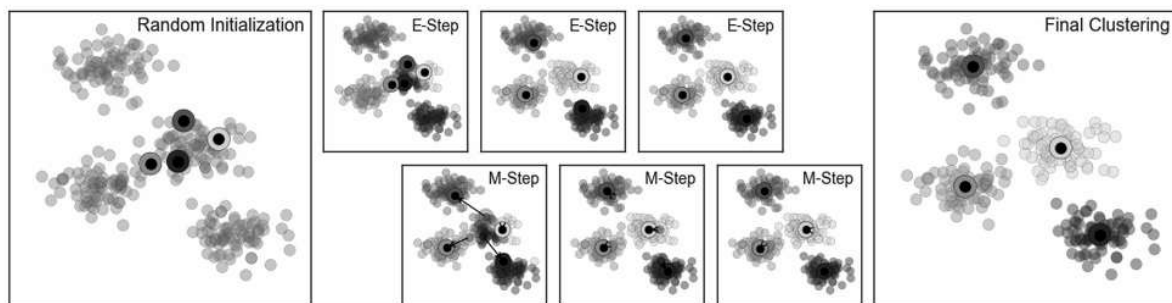


图 5-112: *k*-means 的 E-M 算法的可视化

如图所示，数据从初始化状态开始，经过三次迭代后收敛。下图显示的是聚类的交互式可视化版本，详情请参见 GitHub 在线附录

（<https://github.com/jakevdp/PythonDataScienceHandbook>）中的代码。

k-means 算法非常简单，只要用几行代码就可以实现它。以下是一个非常基础的 *k*-means 算法实现（如图 5-113 所示）：

```
In[5]: from sklearn.metrics import pairwise_distances_argmin

def find_clusters(X, n_clusters, rseed=2):
    # 1. 随机选择簇中心点
    rng = np.random.RandomState(rseed)
    i = rng.permutation(X.shape[0])[:n_clusters]
    centers = X[i]

    while True:
        # 2a. 基于最近的中心指定标签
```

```

labels = pairwise_distances_argmin(X, centers)

# 2b. 根据点的平均值找到新的中心
new_centers = np.array([X[labels == i].mean(0)
                        for i in range(n_clusters)])

# 2c. 确认收敛
if np.all(centers == new_centers):
    break
centers = new_centers

return centers, labels

centers, labels = find_clusters(X, 4)
plt.scatter(X[:, 0], X[:, 1], c=labels,
            s=50, cmap='viridis');

```

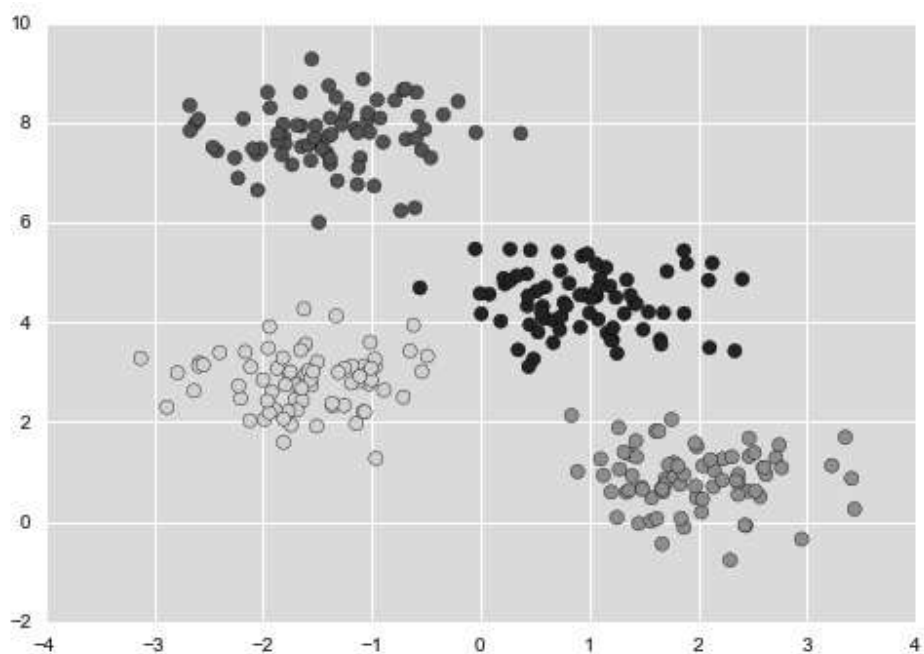


图 5-113: 用 **k-means** 进行数据聚类

虽然大部分可用的聚类算法底层其实都是对上述示例的进一步扩展，但上述函数解释了期望最大化方法的核心内容。

使用期望最大化算法时的注意事项

在使用期望最大化算法时，需要注意几个问题。

可能不会达到全局最优结果

首先，虽然 E-M 算法可以在每一步中改进结果，但是它并不保证可以获得全局最优的解决方案。例如，如果在上述简单的步骤中使用一个随机种子（random seed），那么某些初始值可能会导致很糟糕的聚类结果（如图 5-114 所示）：

```
In[6]: centers, labels = find_clusters(X, 4, rseed=0)
      plt.scatter(X[:, 0], X[:, 1], c=labels,
                  s=50, cmap='viridis');
```

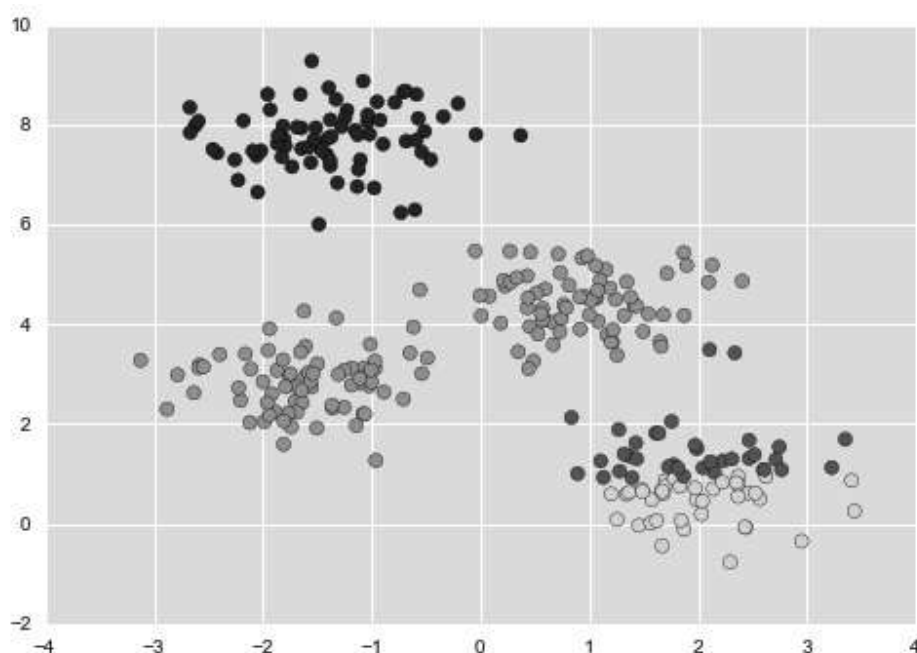


图 5-114: **k-means** 算法的一个糟糕的收敛结果

虽然 E-M 算法最终收敛了，但是并没有收敛至全局最优配置。因此，该算法通常会用不同的初始值尝试很多遍，在 Scikit-Learn 中通过 `n_init` 参数（默认值是 10）设置执行次数。

簇数量必须事先定好

k-means 还有一个显著的问题：你必须告诉该算法簇数量，因为它

无法从数据中自动学习到簇的数量。如果我们告诉算法识别出 6 个簇，它将很快乐地执行，并找出最佳的 6 个簇（如图 5-115 所示）：

```
In[7]: labels = KMeans(6, random_state=0).fit_predict(X)
plt.scatter(X[:, 0], X[:, 1], c=labels,
            s=50, cmap='viridis');
```

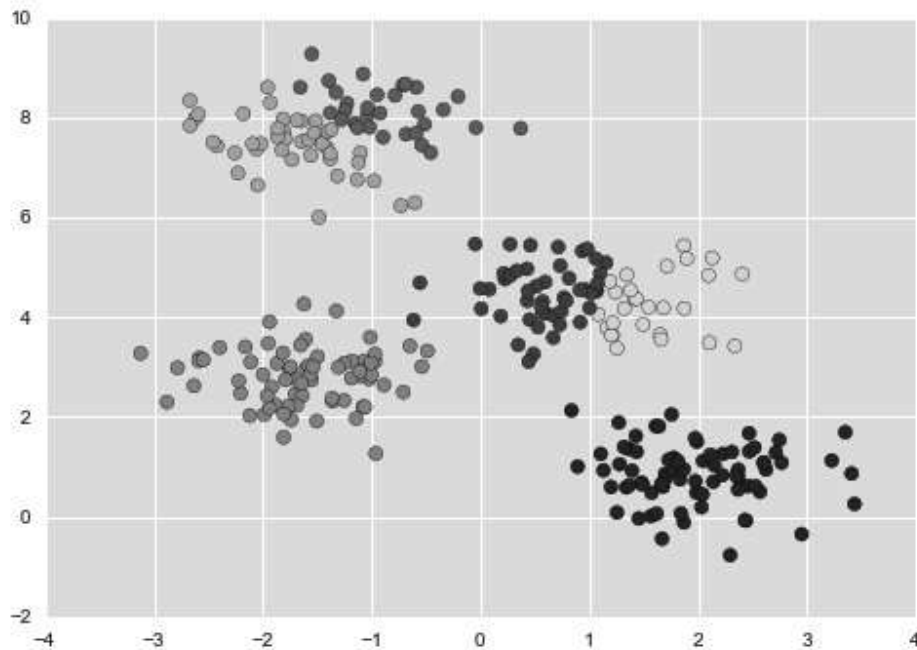


图 5-115：簇数量取值不合适的结果

结果是否有意义是一个很难给出明确回答的问题。有一个非常直观的方法，但这里不会进一步讨论，该方法叫作轮廓分析（http://scikit-learn.org/stable/auto_examples/cluster/plot_kmeans_silhouette_analysis.html）。

不过，你也可以使用一些复杂的聚类算法，有些算法对每个簇的聚类效果有更好的度量方式（例如高斯混合模型，Gaussian mixture models，详情请参见 5.12 节），还有一些算法可以选择一个合适的簇数量（例如 DBSCAN、均值漂移或者近邻传播，这些都是 `sklearn.cluster` 的子模块）。

k-means 算法只能确定线性聚类边界

k-means 的基本模型假设（与其他簇的点相比，数据点更接近自己

的簇中心点) 表明, 当簇中心点呈现非线性的复杂形状时, 该算法通常不起作用。

k -means 聚类的边界总是线性的, 这就意味着当边界很复杂时, 算法会失效。用下面的数据来演示 k -means 算法得到的簇标签, 如图 5-116 所示:

```
In[8]: from sklearn.datasets import make_moons
       X, y = make_moons(200, noise=.05, random_state=0)

In[9]: labels = KMeans(2, random_state=0).fit_predict(X)
       plt.scatter(X[:, 0], X[:, 1], c=labels,
                   s=50, cmap='viridis');
```

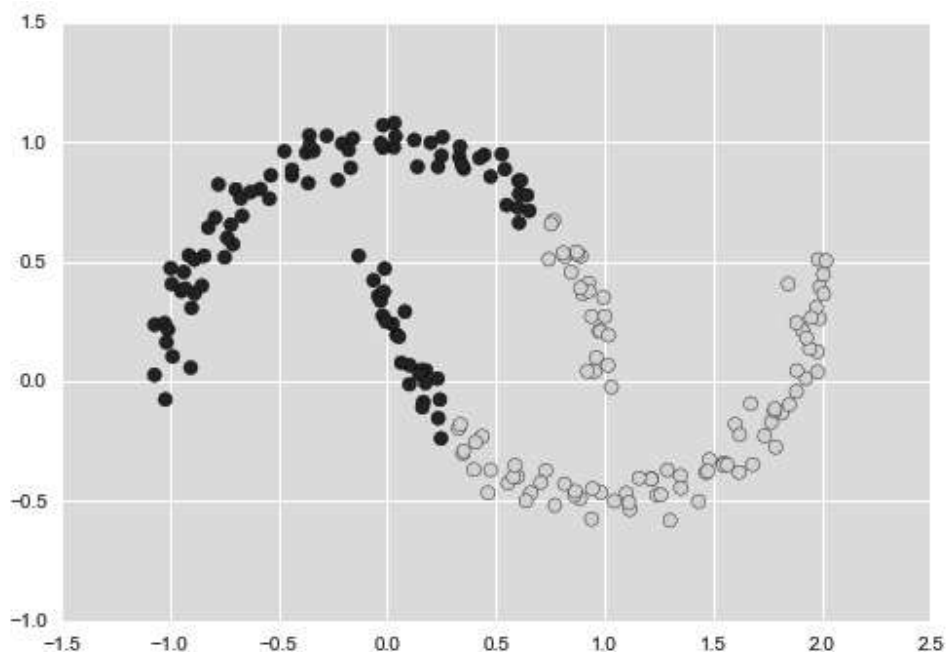


图 5-116: 将 k -means 算法用于非线性边界的失败案例

这个情形让人想起 5.7 节介绍的内容, 当时我们通过一个核变换将数据投影到更高维的空间, 投影后的数据使线性分离成为可能。或许可以使用同样的技巧解决 k -means 算法无法处理非线性边界的问题。

这种核 k -means 算法在 Scikit-Learn 的 `SpectralClustering` 评估器中实现, 它使用最近邻图 (the graph of nearest neighbors) 来计算数据

的高维表示，然后用 *k*-means 算法分配标签（如图 5-117 所示）：

```
In[10]: from sklearn.cluster import SpectralClustering
        model = SpectralClustering(n_clusters=2,
                                   affinity='nearest_neighbors',
                                   assign_labels='kmeans')

        labels = model.fit_predict(X)
        plt.scatter(X[:, 0], X[:, 1], c=labels,
                    s=50, cmap='viridis');
```

可以看到，通过核变换方法，核 *k*-means 就能够找到簇之间复杂的非线性边界了。

当数据量较大时，***k*-means** 会很慢

由于 *k*-means 的每次迭代都必须获取数据集所有的点，因此随着数据量的增加，算法会变得缓慢。你可能会想到将“每次迭代都必须使用所有数据点”这个条件放宽，例如每一步仅使用数据集的一个子集来更新簇中心点。这恰恰就是批处理（batch-based）*k*-means 算法的核心思想，该算法在 `sklearn.cluster.MinibatchKMeans` 中实现。该算法的接口和标准的 `KMeans` 接口相同，后面将用一个示例来演示它的用法。

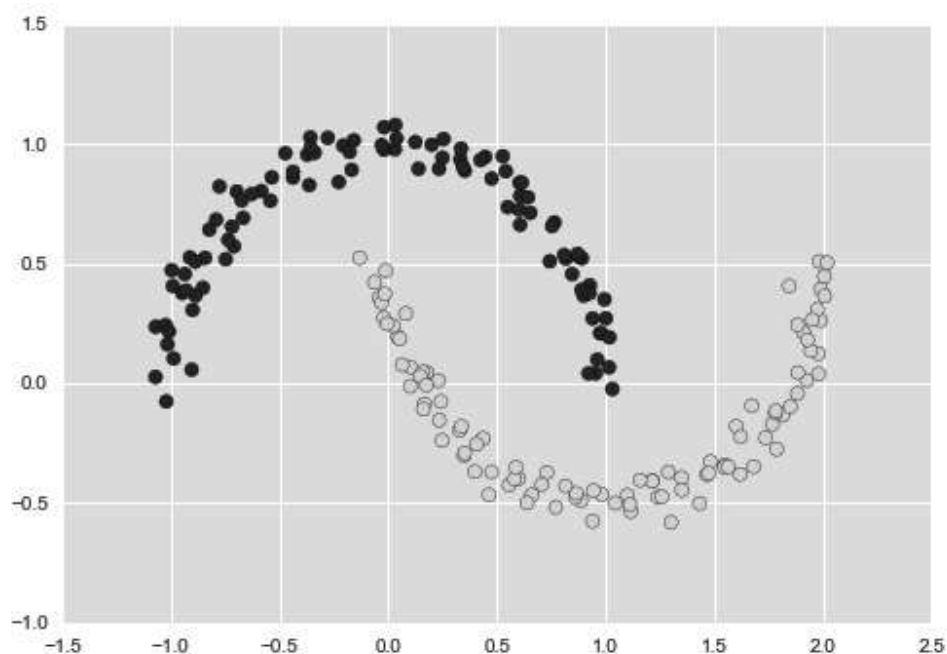


图 5-117: **SpectralClustering** 算法计算的非线性边界

5.11.3 案例

了解了算法的限制条件之后，就可以将 *k*-means 的优势发挥到合适的场景中了。下面来演示一些示例。

01. 案例1：用*k*-means算法处理手写数字

首先，将 *k*-means 算法用于 5.8 节和 5.9 节演示的手写数字数据。这次试试能不能不使用原始的标签信息，就用 *k*-means 算法识别出类似的数字。这个过程就好像是在事先没有标签信息的情况下，探索新数据集的含义。

首先导入手写数字，再使用 **KMeans** 聚类。手写数字数据集包含 1797 个示例，每个样本有 64 个特征，其实就是 8×8 图像中的每个像素：

```
In[11]: from sklearn.datasets import load_digits
        digits = load_digits()
        digits.data.shape

Out[11]: (1797, 64)
```

聚类过程和之前的一样：

```
In[12]: kmeans = KMeans(n_clusters=10, random_state=0)
        clusters = kmeans.fit_predict(digits.data)
        kmeans.cluster_centers_.shape

Out[12]: (10, 64)
```

结果是在 64 维中有 10 个类。需要注意的是，这些簇中心点本身就是 64 维像素的点，可以将这些点看成是该簇中“具有代表性”（typical）的数字。这些簇中心点如图 5-118 所示：

```
In[13]: fig, ax = plt.subplots(2, 5, figsize=(8, 3))
        centers = kmeans.cluster_centers_.reshape(10, 8, 8)
```

```
for axi, center in zip(ax.flat, centers):
    axi.set(xticks=[], yticks=[])
    axi.imshow(center, interpolation='nearest', cmap=plt.cm.bi
```

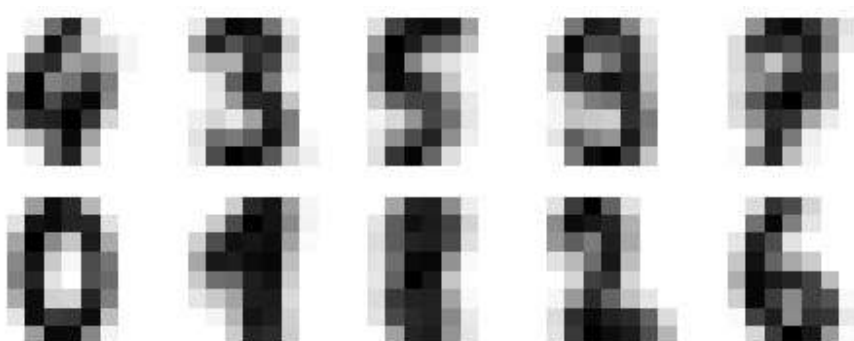


图 5-118: 用 **k-means** 算法得到的簇中心点

我们会发现，即使没有标签，**KMeans** 算法也可以找到可辨识的数字中心，但是 1 和 8 例外。

由于 **k-means** 并不知道簇的真实标签，因此 0~9 标签可能并不是顺序排列的。我们可以将每个学习到的簇标签和真实标签进行匹配，从而解决这个问题：

```
In[14]: from scipy.stats import mode

        labels = np.zeros_like(clusters)
        for i in range(10):
            mask = (clusters == i)
            labels[mask] = mode(digits.target[mask])[0]
```

现在就可以检查无监督聚类算法在查找相似数字时的准确性了：

```
In[15]: from sklearn.metrics import accuracy_score
        accuracy_score(digits.target, labels)

Out[15]: 0.79354479688369506
```

仅通过一个简单的 **k-means** 算法，就可以获得手写数字数据集 80% 的分组准确率！下面再看看混淆矩阵（如图 5-119 所示）：

```
In[16]: from sklearn.metrics import confusion_matrix
        mat = confusion_matrix(digits.target, labels)
        sns.heatmap(mat.T, square=True, annot=True, fmt='d', cbar=False,
                    xticklabels=digits.target_names,
                    yticklabels=digits.target_names)
        plt.xlabel('true label')
        plt.ylabel('predicted label');
```

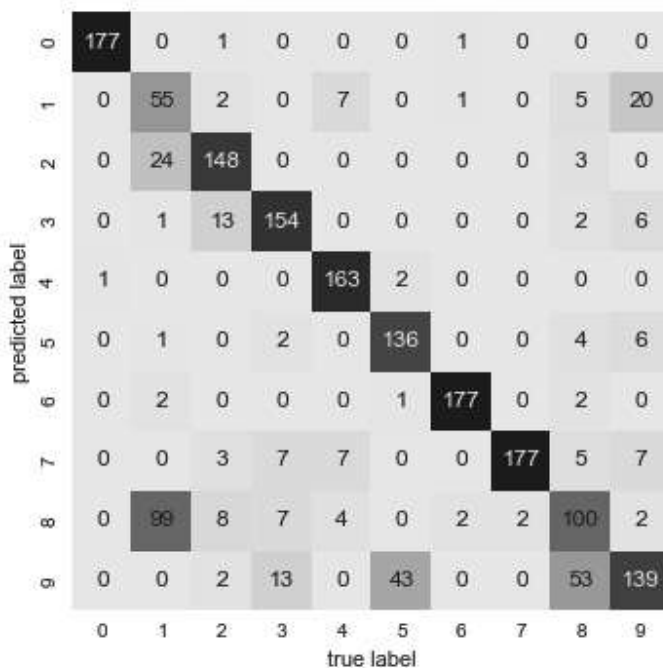


图 5-119: *k-means* 分类器的混淆矩阵

正如我们之前看到的簇中心点图，混淆的地方主要是在数字 8 和 1。但仍然可以看出，通过 *k-means* 可以构建一个数字分类器，该数字分类器不需要任何已知的标签。

其实还可以更进一步，使用 t- 分布邻域嵌入算法（详情请参见 5.10 节）在执行 *k-means* 之前对数据进行预处理。t-SNE 是一个非线性嵌入算法，特别擅长保留簇中的数据点。下面来看看如何实现：

```
In[17]: from sklearn.manifold import TSNE

        # 投影数据：这一步将耽误几秒钟
        tsne = TSNE(n_components=2, init='pca', random_state=0)
        digits_proj = tsne.fit_transform(digits.data)
```

```
# 计算类
kmeans = KMeans(n_clusters=10, random_state=0)
clusters = kmeans.fit_predict(digits_proj)

# 排列标签
labels = np.zeros_like(clusters)
for i in range(10):
    mask = (clusters == i)
    labels[mask] = mode(digits.target[mask])[0]

# 计算准确度
accuracy_score(digits.target, labels)

Out[17]: 0.93356149137451305
```

同样在没有标签的情况下，它可以达到 94% 的分类准确率，这就是合理使用无监督学习的力量。无监督学习可以从数据集中抽取难以用手眼直接提取的信息。

02. 案例2：将**k-means**用于色彩压缩

聚类算法的另一个有趣应用是图像色彩压缩。设想你有一幅包含几百万种颜色的图像，但其实大多数图像中的很大一部分色彩通常是不会被眼睛注意到的，而且图像中的很多像素都拥有类似或者相同的颜色。

如图 5-120 所示，该图像来源于 Scikit-Learn 的 **datasets** 模块（在本例中，你将需要安装 Python 的 **pillow** 图像程序包）：

```
In[18]: # 需要安装pillow图像程序包
from sklearn.datasets import load_sample_image
china = load_sample_image("china.jpg")
ax = plt.axes(xticks=[], yticks=[])
ax.imshow(china);
```



图 5-120: 输入图像

该图像存储在一个三维数组 (`height`, `width`, `RGB`) 中, 以 0~255 的整数表示红 / 蓝 / 绿信息:

```
In[19]: china.shape  
Out[19]: (427, 640, 3)
```

可以将这组像素转换成三维颜色空间中一群数据点。先将数据变形为 `[n_samples × n_features]`, 然后缩放颜色至其取值为 0~1:

```
In[20]: data = china / 255.0 # 转换成0~1区间值  
        data = data.reshape(427 * 640, 3)  
        data.shape  
Out[20]: (273280, 3)
```

还可以在颜色空间中对这些像素进行可视化。为了演示方便, 这里只使用包含前 10 000 个像素的子集 (如图 5-121 所示):

```
In[21]: def plot_pixels(data, title, colors=None, N=10000):
```

```

if colors is None:
    colors = data

# 随机选择一个子集
rng = np.random.RandomState(0)
i = rng.permutation(data.shape[0])[:N]
colors = colors[i]
R, G, B = data[i].T

fig, ax = plt.subplots(1, 2, figsize=(16, 6))
ax[0].scatter(R, G, color=colors, marker='.')
ax[0].set(xlabel='Red', ylabel='Green', xlim=(0, 1), ylim=(0, 1))

ax[1].scatter(R, B, color=colors, marker='.')
ax[1].set(xlabel='Red', ylabel='Blue', xlim=(0, 1), ylim=(0, 1))

fig.suptitle(title, size=20);

```

```
In[22]: plot_pixels(data, title='Input color space: 16 million possible colors')
```

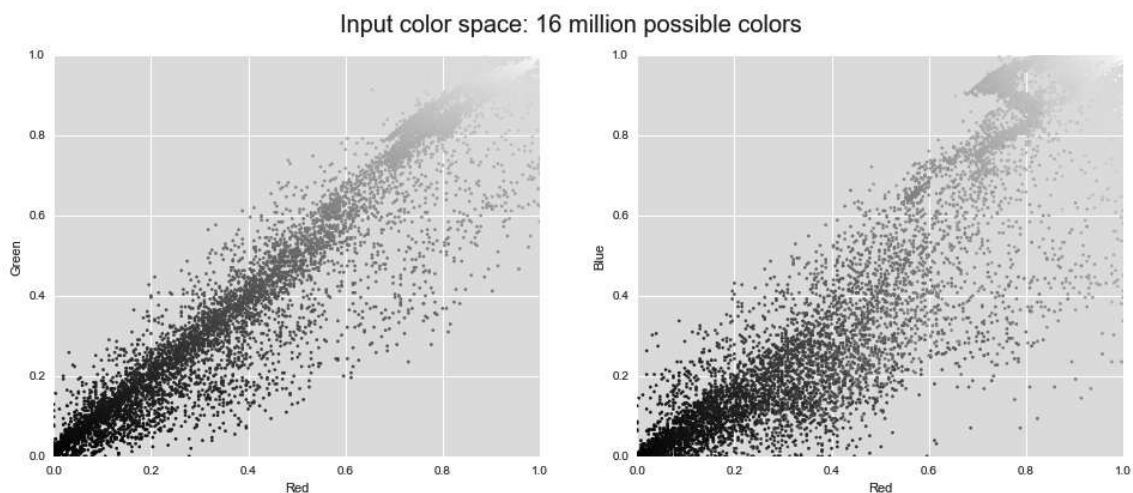


图 5-121: 在 **RGB** 颜色空间中的像素分布

现在对像素空间（特征矩阵）使用 *k*-means 聚类，将 1600 万种颜色（ $255 \times 255 \times 255 = 16\,581\,375$ ）缩减到 16 种颜色。因为我们处理的是一个非常大的数据集，所以将使用 **MiniBatchKMeans** 算法对数据集的子集进行计算。这种算法比标准的 *k*-means 算法速度更快（如图 5-122 所示）：

```
In[23]: from sklearn.cluster import MiniBatchKMeans
```

```

kmeans = MiniBatchKMeans(16)
kmeans.fit(data)
new_colors = kmeans.cluster_centers_[kmeans.predict(data)]

plot_pixels(data, colors=new_colors,
            title="Reduced color space: 16 colors")

```

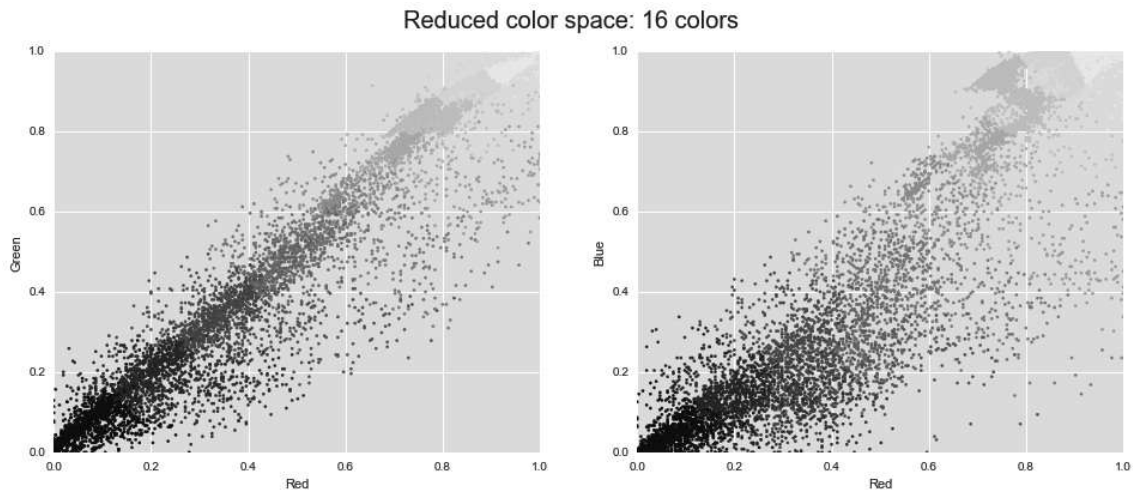


图 5-122: 在 RGB 颜色空间中 16 个类

用计算的结果对原始像素重新着色，即每个像素被指定为距离其最近的簇中心点的颜色。用新的颜色在图像空间（ 427×640 ），而不是像素空间（ $273\ 280 \times 3$ ）里重新画图，展示效果（如图 5-123 所示）：

```

In[24]:
china_recolored = new_colors.reshape(china.shape)

fig, ax = plt.subplots(1, 2, figsize=(16, 6),
                      subplot_kw=dict(xticks=[], yticks=[]))
fig.subplots_adjust(wspace=0.05)
ax[0].imshow(china)
ax[0].set_title('Original Image', size=16)
ax[1].imshow(china_recolored)
ax[1].set_title('16-color Image', size=16);

```




图 5-123: 对比拥有全部颜色的图像（左）和拥有 **16** 个颜色的图像（右）

虽然右图丢失了某些细节，但是图像总体上还是非常容易辨识的。右图实现了将近一百万的压缩比！这就是 *k-means* 的一个有趣的应用，当然还有很多更好的压缩图像的算法，但是这个示例足以显示无监督算法（如 *k-means*）解决问题的力量。